

XGBoost (Extreme Gradient Boosting) — Detaylı Konu Anlatımı

Sınav Çalışma Notları

Contents

1. XGBoost Nedir? Tarihçe ve Motivasyon	1
2. Matematiksel Temel: İkinci Derece Taylor Açılımı	2
3. Regularizasyon Terimi	2
4. Ağaç Budama (Pruning) — Farklı Bir Yaklaşım	2
5. Ağırlıklı Nicel Özetleme (Weighted Quantile Sketch)	3
6. Eksik Veri Yönetimi (Sparsity-Aware Split Finding)	3
7. Parallelleştirme — Neyin Paralel, Neyin Sıralı Olduğu	3
8. Erken Durdurma (Early Stopping)	3
9. Önemli Hiperparametreler — Sınav İçin Özet Tablo	4
10. XGBoost'un Diğer Yöntemlerle Karşılaştırması	4
11. Kullanım Alanı ve Pratik Notlar	5
12. Sınavda Sık Sorulan Kavramlar — Hızlı Hatırlatma	5

1. XGBoost Nedir? Tarihçe ve Motivasyon

XGBoost (eXtreme Gradient Boosting), Tianqi Chen tarafından 2016'da geliştirilen, klasik Gradient Boosting algoritmasının **regularize edilmiş, ikinci derece optimizasyon kullanan ve büyük ölçekte çalışacak şekilde mühendislik açısından optimize edilmiş** bir versiyonudur. Kaggle yarışmalarında sıkça birinci olan modellerin ortak paydası olması nedeniyle "yapılandırılmış (tabular) veri için altın standart" olarak anılır.

Klasik Gradient Boosting'in Sınırlamaları (Hatırlatma): Regularizasyon eksikliği, yerleşik erken durdurma olmaması, büyük veride yavaşlık, eksik veriyi otomatik yönetememe. XGBoost bu dört sorunu doğrudan hedef alır.

2. Matematiksel Temel: İkinci Derece Taylor Açılımı

Klasik Gradient Boosting sadece **birinci türevi (gradyanı)** kullanırken, XGBoost kayıp fonksiyonunun **ikinci dereceden Taylor açılımını** kullanır:

$$L^{(t)} \approx \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t)$$

Burada:

- $g_i = \partial_{\hat{y}} L(y_i, \hat{y}^{(t-1)}) \rightarrow$ **gradyan** (birinci türev),
- $h_i = \partial_{\hat{y}}^2 L(y_i, \hat{y}^{(t-1)}) \rightarrow$ **Hessian** (ikinci türev),
- $\Omega(f_t) \rightarrow$ **regularizasyon terimi**.

Neden Önemli? İkinci türev bilgisi, optimizasyonun **yönünü değil aynı zamanda eğril-iğini (curvature)** de kullanmasını sağlar — bu, klasik gradyan inişine göre **daha hızlı ve daha doğru yakınsama** anlamına gelir (Newton yönteminin boosting’e uyarlanmış hâli gibi düşünülebilir).

3. Regularizasyon Terimi

XGBoost’un en ayırt edici özelliği, ağaç karmaşıklığına doğrudan ceza uygulamasıdır:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

Burada:

- $T \rightarrow$ ağaçtaki yaprak sayısı,
- $w_j \rightarrow j$. yaprağın tahmin ağırlığı,
- γ (gamma) $\rightarrow$ yeni bir yaprak eklemenin “maliyeti” (bölünme eşiği),
- λ (reg_lambda) $\rightarrow$ yaprak ağırlıklarının büyüklüğüne uygulanan L2 cezası.

Ayrıca reg_alpha parametresi ile **L1 regularizasyon** da eklenebilir (yaprak ağırlıklarını sıfıra çekerek seyreklik/sparsity sağlar).

Etkisi: Bu terimler sayesinde XGBoost, çok fazla yaprağa sahip aşırı karmaşık ağaçlar kurmak-tan **doğal olarak kaçınır** — klasik Gradient Boosting’de bu kontrol yalnızca max_depth gibi dışsal sınırlamalarla sağlanabiliyordu.

4. Ağaç Budama (Pruning) — Farklı Bir Yaklaşım

Klasik karar ağaçları **“greedy” (açgözlü) durdurma** stratejisi kullanır: bir bölünme, anlık bilgi kazancı negatifse hiç yapılmaz. XGBoost ise farklı çalışır:

1. Ağaç, max_depth’e ulaşana kadar **büyütülür** (bazı dallar anlık olarak kötü görünse bile).
2. Ağaç tamamlandıktan sonra, **geriye doğru (bottom-up)** budama yapılır: bir bölünmenin sağladığı kazanç gamma eşiğinin altındaysa o dal budanır.

Neden Bu Daha İyi? Bazen “kötü” bir bölünme, kendisinden sonra gelen bölünmelerin çok değerli olmasını sağlayabilir (yani anlık kazanç düşük olsa da, o dalın devamı çok bilgi taşıyabilir). Greedy durdurma bu tür durumları kaçırmakla, XGBoost’un “önce büyüt sonra buda” stratejisi bu fırsatları yakalayabilir.

5. Ağırlıklı Nicel Özetleme (Weighted Quantile Sketch)

Büyük veri setlerinde her olası bölünme noktasını tek tek denemek maliyetlidir. XGBoost, sürekli değişkenler için **yaklaşık (approximate) bölünme noktası önerileri** üretir:

- Veri, her örneğin Hessian değeriyle **ağırlıklandırılmış** nicelik (quantile) özetlerine göre kutulara (bucket) ayrılır.
 - Bölünme araması, tüm benzersiz değerler yerine bu **öneri noktaları** üzerinden yapılır.
 - Bu, büyük veri setlerinde bölünme aramasını önemli ölçüde hızlandırır (`tree_method='hist'` veya `'approx'`).
-

6. Eksik Veri Yönetimi (Sparsity-Aware Split Finding)

XGBoost, eksik (NaN) değerleri **özel bir şekilde** ele alır:

1. Eğitim sırasında, her bölünme için eksik değerlerin **hangi yöne (sol/sağ)** gönderilmesi gerektiği **veriden otomatik olarak öğrenilir** (hangi yön kaybı daha çok azaltıyorsa o yön seçilir).
2. Tahmin sırasında, eksik bir değerle karşılaşıldığında, öğrenilen bu **varsayılan yön** kullanılır.

Pratik Sonuç: Kullanıcı, eksik değerleri manuel olarak doldurmak (imputation) zorunda kalmadan XGBoost’u doğrudan eksik verili veri setleri üzerinde çalıştırabilir — bu, gerçek dünya (özellikle finansal/dolandırıcılık) veri setlerinde büyük bir pratik avantajdır.

7. Parallelleştirme — Neyin Paralel, Neyin Sıralı Olduğu

Sık Yapılan Kavram Yanılgısı: “XGBoost ağaçları paralel kurar” **YANLIŞTIR**. Boosting’in doğası gereği, her ağaç bir öncekinin hatasına dayandığı için **ağaçların kendisi hâlâ sıralı (sequential) olarak kurulur**.

Peki Ne Parallelleştirilir? Her ağacın **içindeki** bölünme arama süreci paralelleştirilir:

- Özellikler önceden sıralanıp (pre-sorted) bloklar halinde saklanır (**column block** yapısı).
- Farklı özellikler için en iyi bölünme noktası araması **eş zamanlı (multi-threaded)** yapılabilir.

Bu, “ağaç düzeyinde sıralı, bölünme arama düzeyinde paralel” şeklinde özetlenebilir.

8. Erken Durdurma (Early Stopping)

XGBoost’ta yerleşik olarak bulunan bu mekanizma:

1. Eğitim sırasında bir **doğrulama seti (eval_set)** belirlenir.
2. Her iterasyondan sonra doğrulama metriği (örn. AUC) hesaplanır.
3. Eğer belirlenen sayıda (early_stopping_rounds) iterasyon boyunca doğrulama metriği **iyileşmezse**, eğitim otomatik olarak durur ve en iyi iterasyondaki model kullanılır.

Faydası: n_estimators'ı çok yüksek bir üst sınır olarak verip, gerçek optimum ağaç sayısını algoritmanın kendisinin bulmasına izin verir — bu hem overfitting'i önler hem de gereksiz hesaplamayı engeller.

9. Önemli Hiperparametreler — Sınav İçin Özet Tablo

Parametre	Anlamı	Etkisi
n_estimators	Ağaç sayısı	Erken durdurma ile birlikte kullanılırsa üst sınır işlevi görür
learning_rate (eta)	Küçültme katsayısı	Düşükse daha fazla ağaç gerekir, genelde daha iyi genelleme
max_depth	Ağaç derinliği	Genelde 3-10 arası; regularizasyon sayesinde RF'ye göre biraz daha derin tutulabilir
gamma	Bölünme için gereken min. kayıp azalması	Artınca ağaç daha az dallanır (daha muhafazakâr)
reg_alpha	L1 regularizasyon	Seyreklik sağlar, gereksiz özellikleri sıfırlar
reg_lambda	L2 regularizasyon	Yaprak ağırlıklarını küçültür, overfitting'i azaltır
subsample	Satır örnekleme oranı	<1.0 iken varyansı azaltır (Stochastic GB gibi)
colsample_bytree	Sütun örnekleme oranı	Random Forest'taki max_features'a benzer mantık
scale_pos_weight	Dengesiz sınıflandırmada pozitif sınıf ağırlığı	Azınlık sınıfın önemini artırır
tree_method	Bölünme arama algoritması	'hist' büyük veri setlerinde hız için tercih edilir

10. XGBoost'un Diğer Yöntemlerle Karşılaştırması

Kriter	Gradient Boosting (klasik)	XGBoost
Optimizasyon derecesi	Birinci derece (gradyan)	İkinci derece (gradyan + Hessian)
Regularizasyon	Yok/sınırlı	L1 + L2 (yerleşik)

Kriter	Gradient Boosting (klasik)	XGBoost
Ağaç budama	Greedy (anlık durdurma)	Önce büyüt, sonra buda (gamma ile)
Eksik veri Erken durdurma	Manuel doldurma gerekir Yerleşik değil	Otomatik yön öğrenimi Yerleşik (early_stopping_rounds)
Hız/ölçeklenebilirlik	Orta	Yüksek (histogram/blok tabanlı paralelleştirme)

XGBoost vs LightGBM: İkisi de Gradient Boosting'in gelişmiş versiyonlarıdır; temel fark **ağaç büyütme stratejisidir** (XGBoost varsayılan olarak level-wise/katman katman büyütür, LightGBM leaf-wise/en iyi yaprak önce büyütür) ve **hız optimizasyon teknikleridir** (LightGBM'in GOSS ve EFB teknikleri — detaylar LightGBM konu anlatımında).

11. Kullanım Alanı ve Pratik Notlar

XGBoost, özellikle şu senaryolarda tercih edilir:

- **Yapılandırılmış/tablo (tabular) veri** — görüntü/metin gibi yapılandırılmamış veride derin öğrenme genelde daha üstündür.
- **Dengesiz sınıflandırma problemleri** (dolandırıcılık tespiti, kredi riski) — `scale_pos_weight` ile kolayca uyarlanabilir.
- **Orta-büyük ölçekli veri setleri** — çok daha büyük veride (milyonlarca-milyarlarca satır) LightGBM genelde daha hızlıdır.
- **Eksik veri içeren gerçek dünya veri setleri** — otomatik yön öğrenimi sayesinde ön işleme yükü azalır.

12. Sınavda Sık Sorulan Kavramlar — Hızlı Hatırlatma

- XGBoost = Gradient Boosting + **regularizasyon (L1/L2) + ikinci derece optimizasyon (Hessian) + mühendislik optimizasyonları**.
- **Ağaçlar hâlâ sıralı kurulur** — paralelleştirilen şey, her ağacın içindeki **bölünme arama sürecidir**, ağaçların kendisi değil.
- gamma, bir bölünmenin gerçekleşmesi için gereken minimum kayıp azalmasıdır; büyütüp sonra budama stratejisiyle çalışır.
- Eksik değerler için **yön otomatik öğrenilir** — manuel imputation gerekmez.
- `scale_pos_weight`, dengesiz sınıflandırmada pozitif (azınlık) sınıfın ağırlığını artırır.
- Erken durdurma (`early_stopping_rounds`), doğrulama performansı iyileşmeyince eğitimi otomatik durdurur.
- `tree_method='hist'`, histogram tabanlı yaklaşık bölünme aramasıyla büyük veri setlerinde belirgin hız kazancı sağlar.